

Informe Laboratorio 9 – Diseño de Controladores con Utility System

Diego Osvaldo López Castillo

15 de mayo de 2026

1. Introducción

En este laboratorio se crearon controladores basados en **Utility Systems** para **Ms. Pacman** y el fantasma **Clyde/Sue**.

Este método consiste en asignar **puntajes de utilidad** a distintos comportamientos y seleccionar aquella con **mayor utilidad**.

A diferencia de métodos como las **FSM**, donde se realiza una transición entre estados, en los **Utility System** se evalúan constantemente las acciones mediante **funciones matemáticas**, lo cual permite decisiones más flexibles y dinámicas.

Los comportamientos implementados fueron:

1.1. Ms. Pacman

- Escapar de fantasmas peligrosos.
- Perseguir fantasmas comestibles.
- Buscar píldoras.

1.2. Clyde

- Patrullar un territorio.
- Perseguir a Ms. Pacman cuando se acerca a su zona.
- Escapar cuando se encuentra en estado comestible.

2. Ms. Pacman

2.1. Comportamientos

Este controlador evalúa tres comportamientos distintos.

- Escapar de fantasmas peligrosos.
- Perseguir fantasmas comestibles.

- Buscar/Comer píldoras.

Estos comportamientos van adquiriendo una utilidad distinta y posteriormente con comparadas por el controlador para decidir qué hacer.

2.2. Movimiento hacia objetivos

Para poder moverse por el mapa se implementaron dos funciones: `getClosestMove()` y `getFarthestMove()`.

```
class MsPacmanUtilitySys: public Controller
{
private:
    Move getClosestMove(const GameState& game, std::pair<int,int> target) const;
    Move getFarthestMove(const GameState& game, std::pair<int,int> target) const;
```

Figura 1: Funciones de movimiento

La primera selecciona el movimiento que deja a Ms. Pacman **más cerca de un objetivo**, mientras que la segunda selecciona el **movimiento que la aleja más**. Ambas funciones revisan los movimientos legales disponibles y calculan la distancia euclidiana entre cada posible posición y el objetivo correspondiente. La distancia utilizada fue:

```
float dist = euclid2(pos, target);
```

Figura 2: Distancia entre posición y el objetivo.

2.3. Escape de fantasmas

El comportamiento de escape se activa cuando un **fantasma no es comestible** y se encuentra **cerca**. Para calcular el peligro se utilizó una **curva logística**:

$$U_{fear} = \frac{1}{1 + e^{(d-30) \cdot 0,15}}$$

Donde:

- d es la distancia entre Ms. Pacman y el fantasma.
- Mientras menor sea la distancia, mayor será la utilidad de escapar.

Esta función fue elegida porque permite que el aumento del peligro ocurra de forma progresiva. Cuando el fantasma está lejos, la utilidad es baja, pero al acercarse aumenta rápidamente. Si la utilidad de miedo es la más alta, Ms. Pacman utiliza `getFarthestMove()` para **alejarse** del **fantasma más cercano**.

2.4. Persecución de fantasmas comestibles

Cuando un fantasma entra en estado edible, Ms. Pacman puede perseguirlo para obtener puntos. La utilidad utilizada fue una **función cuadrática**:

$$U_{hunt} = \frac{\text{máx}(0, 80 - d)^2}{80^2}$$

Esta función entrega mayor utilidad a fantasmas cercanos y reduce rápidamente el interés por perseguir fantasmas lejanos. Cuando esta utilidad es la más alta, Ms. Pacman utiliza `getClosestMove()` para **acercarse** al fantasma comestible más conveniente.

2.5. Búsqueda de píldoras

El comportamiento por defecto de Ms. Pacman consiste en buscar la píldora más cercana. La utilidad utilizada fue:

$$U_{pill} = \frac{0,4}{1 + d \cdot 0,02}$$

donde:

- d corresponde a la distancia a la píldora más cercana.

La utilidad disminuye gradualmente mientras la píldora está más lejos. Este comportamiento actúa como comportamiento base para Ms. Pacman

2.6. Selección final

Después de calcular todas las utilidades, el controlador compara los valores:

- Si el **miedo** es **mayor**, Ms. Pacman **escapa**.
- Si la **utilidad** de **cazar fantasmas** es **mayor**, los persigue.
- En cualquier otro caso busca píldoras.

3. Clyde

3.1. Comportamientos

Se diseñó un comportamiento de un fantasma territorial, el cual patrulla una zona específica. Sus comportamientos principales son:

- Patrullar una zona específica.
- Perseguir a Ms. Pacman si se acerca.
- Escapar cuando se encuentra vulnerable.

3.2. Patrullaje

Se definieron varios puntos de patrulla ubicados en la esquina inferior izquierda del mapa.

```
//Zona inferior izquierda
patrolPoints.push_back({20, 650});
patrolPoints.push_back({120, 650});
patrolPoints.push_back({20, 520});
patrolPoints.push_back({120, 520});
```

Figura 3: Puntos de patrullaje

Clyde recorre estos puntos secuencialmente utilizando la función `getClosestMove()`, la cual es similar a la usada para Ms. Pacman.

Cuando el fantasma se acerca lo suficiente a un punto, cambia automáticamente al siguiente.

Esto genera un movimiento constante dentro de su territorio.

La utilidad de patrulla se dejó fija en:

$$patrolUtility = 0,2$$

ya que este comportamiento representa el estado normal del fantasma.

3.3. Persecución dentro del territorio

Para representar un comportamiento territorial, Clyde calcula la distancia entre Ms. Pacman y el centro de su zona de movimiento.

```
territoryCenter = {70, 580};
```

Figura 4: Centro del territorio

La utilidad de persecución se calculó normalizando esa distancia:

$$d_n = \min\left(\frac{d}{600}, 1\right)$$

Luego se utilizó:

$$U_{chase} = 1 - d_n$$

Con este enfoque:

- Cuando Ms. Pacman está lejos del territorio, la utilidad de persecución es baja.
- Mientras más se acerca, la utilidad aumenta.

Si la **utilidad de persecución** supera la utilidad de patrulla, Clyde comienza a perseguir a Ms. Pacman utilizando `getClosestMove()`.

Si la utilidad de persecución se reduce por debajo de la utilidad de patrulla, entonces el fantasma volverá a su esquina asignada para patrullar.

3.4. Escape de Clyde

Cuando el fantasma ingresa en estado `.edible`, se activa automáticamente el comportamiento de escape:

$$U_{escape} = 1,0$$

Esto asegura que la prioridad máxima sea **alejarse** de Ms. Pacman.

Para escapar se utiliza `getFarthestMove()`, buscando maximizar la distancia respecto al jugador.

```
//Estado frightened
if(ghost->isEdible())
{
    fleeUtility = 1.0f;
}
```

Figura 5: Utilidad en estado Frigthened

4. Conclusión

El uso de Utility Systems permitió implementar comportamientos más flexibles y dinámicos tanto para Ms. Pacman como para Clyde.

Ms. Pacman puede decidir entre escapar, atacar o comer píldoras.

Por otro lado, **Clyde** combina patrullaje, comportamiento territorial y huida en situaciones de peligro.

Las curvas de utilidad utilizadas permitieron controlar cómo cambia la prioridad de cada comportamiento según la distancia y el estado del juego, logrando obtener comportamientos más suaves y naturales.